

Question 2 - Models / Regression / Classification Exam Templates

This sheet collects the recurring **model questions** from the 2024-2026 exam material we reviewed. The exact story changes, but the same branches repeat: **linear regression**, **logistic classification**, **threshold/cost optimisation**, **calibration**, **0-1 loss**, **precision/recall**, **MARE/MSE/MAE**, **DKW/ECDF**, and **Hoeffding confidence intervals**.

The main rule is the same as for SVD:

Identify what kind of target you have first, then read the later subquestions to see which model branch to attach.

0. Fast story-to-method map

Question wording	Think immediately
target is salary / time / price / continuous number	Linear Regression
target is 0/1, spam/not spam, fraud/not fraud, disease/no disease	Logistic / binary classification
<code>predict_proba</code> , probability of class 1	Logistic probability output
“threshold” / cutoff / classify positive if probability $\geq t$	Custom classification threshold
false positive / false negative have different costs	Cost-sensitive classification
“minimize average cost”	Search thresholds + <code>argmin(costs)</code>
“minimize 0-1 loss”	Search thresholds + <code>argmin(classification_error)</code>
“precision / recall”	Confusion-matrix metrics
“calibration set” / calibrate probabilities	Calibration model after classifier
“0-1 loss”	Mean of wrong predictions
“baseline always predicts training mean”	Regression baseline
“MSE / MAE / MARE”	Regression evaluation
“residual”	<code>y_true - y_pred</code>
“predicted vs true scatter”	Scatter plot + ideally diagonal reference
“ECDF with confidence bands”	DKW inequality / EDF
“bounded loss/cost + confidence interval”	Hoeffding
“95% confidence”	<code>delta = 0.05</code>
“99% confidence”	<code>delta = 0.01</code>

PART A - COMMON START: LOAD X AND y

1. Load a dataset and choose features/target

```
import numpy as np
import pandas as pd

# Load data
df = pd.read_csv("data.csv")

# Example: choose feature columns
features = ["x1", "x2", "x3"]
target = "y"

X = df[features].to_numpy(dtype=float)
y = df[target].to_numpy()
```

Story translation

If the question says:

“Predict salary using work year, experience level and remote ratio.”

then:

```
features = ["work_year", "experience_level", "remote_ratio"]
target = "salary_in_usd"
```

If the question says:

“Predict fraud where Class is 0/1.”

then:

```
X = df[["V1", "V2", "V3", "V4", "Amount"]].to_numpy()
y = df["Class"].to_numpy()
```

PART B - TRAIN / TEST / VALIDATION SPLITS

2. Standard train/test split

Trigger: “80% training, 20% testing”, “train size 0.8”, “held-out test set”.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    train_size=0.8,
    random_state=42
)
```

If the exam explicitly says **first 80% / last 20%**, do NOT use random splitting:

```
split = int(0.8 * len(X))
```

```
X_train = X[:split]
X_test  = X[split:]
y_train = y[:split]
y_test  = y[split:]
```

3. Train / validation / test split

Trigger: “training, validation and testing”, “choose threshold on validation, evaluate on test”.

```
from sklearn.model_selection import train_test_split

X_train, X_tmp, y_train, y_tmp = train_test_split(
    X, y,
    train_size=0.6,
    random_state=42
)

X_val, X_test, y_val, y_test = train_test_split(
    X_tmp, y_tmp,
    train_size=0.5,
    random_state=42
)
```

Memory

- **TRAIN** = fit model
- **VALIDATION** = choose threshold / model setting

- **TEST** = final honest evaluation
-

PART C - STANDARDIZATION

4. Standardize using TRAINING statistics only

Triggers:

- “mean 0 and standard deviation 1”
- “statistics estimated only from training/fitting observations”
- “do not use held-out observations”

```
mu = np.mean(X_train, axis=0)
sd = np.std(X_train, axis=0)
```

```
# avoid division by zero
sd[sd == 0] = 1.0
```

```
X_train_std = (X_train - mu) / sd
X_test_std  = (X_test - mu) / sd
```

If there is validation data:

```
X_val_std = (X_val - mu) / sd
```

Exam explanation

I estimated the mean and standard deviation using the training data only, then used the same values to transform the validation/test data. This avoids leakage from held-out observations.

PART D - LINEAR REGRESSION

5. Basic Linear Regression template

Triggers: continuous target such as salary, delivery time, electricity use, house price.

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

With standardized features:

```
model.fit(X_train_std, y_train)
y_pred = model.predict(X_test_std)
```

If coefficients are requested:

```
beta = np.r_[model.intercept_, model.coef_]
print(beta)
```

6. Residuals

Trigger: “residual”, “prediction error”.

```
residuals = y_test - y_pred
```

Remember:

```
residual = true - predicted
```

7. MSE, MAE and R^2

```
mse = np.mean((y_test - y_pred)**2)
mae = np.mean(np.abs(y_test - y_pred))

ss_res = np.sum((y_test - y_pred)**2)
ss_tot = np.sum((y_test - np.mean(y_test))**2)
r2 = 1 - ss_res / ss_tot

print("MSE:", mse)
print("MAE:", mae)
print("R2:", r2)
```

Memory

- MSE lower = better
 - MAE lower = better
 - R^2 higher = better
-

8. MARE - Mean Absolute Relative Error (2025-style)

Trigger:

“mean absolute relative error”

Formula:

```
|true - predicted| / |true|
```

Code:

```
relative_errors = np.abs((y_test - y_pred) / y_test)
mare = np.mean(relative_errors)
```

```
print("MARE:", mare)
```

If zero targets are possible, protect against division by zero:

```
mask = y_test != 0
mare = np.mean(
    np.abs((y_test[mask] - y_pred[mask]) / y_test[mask])
)
```

Interpretation

If MARE = 0.18, then the predictions are off by about **18% on average relative to the true value**.

9. Baseline model - always predict training mean

Trigger:

“compare to a simple predictor that always predicts the average response in the training/fitting sample.”

```
baseline_value = np.mean(y_train)
```

```
baseline_pred = np.full(
    len(y_test),
    baseline_value,
    dtype=float
)
```

```
baseline_mse = np.mean((y_test - baseline_pred)**2)
model_mse = np.mean((y_test - y_pred)**2)
```

```
print("Model MSE:", model_mse)
print("Baseline MSE:", baseline_mse)
```

Interpretation

```
if model_mse < baseline_mse:
    print("Regression model is better")
else:
    print("Baseline is better")
```

MSE lower = better.

10. Predicted vs true scatter plot

Trigger:

“plot predicted value on x-axis and true value on y-axis.”

```
import matplotlib.pyplot as plt

plt.scatter(y_pred, y_test)
plt.xlabel("Predicted value")
plt.ylabel("True value")
plt.title("Predicted vs True")

# optional ideal-reference line
low = min(y_pred.min(), y_test.min())
high = max(y_pred.max(), y_test.max())
plt.plot([low, high], [low, high])

plt.show()
```

Exam explanation

A good regression model should produce points close to the diagonal line `true = predicted`. Large vertical deviations indicate large prediction errors.

PART E - ECDF / DKW FOR REGRESSION RESIDUALS

11. ECDF of residuals

```
residuals = y_test - y_pred

sorted_residuals = np.sort(residuals)
F_n = np.arange(1, len(sorted_residuals) + 1) / len(sorted_residuals)

plt.step(sorted_residuals, F_n, where="post")
plt.xlabel("Residual")
plt.ylabel("ECDF")
plt.show()
```

12. DKW confidence bands - 99% confidence (2025-style)

For confidence level $1-\delta$, DKW gives a uniform EDF band with radius:

```
delta = 0.01      # 99% confidence
n = len(residuals)

epsilon = np.sqrt(
    np.log(2 / delta) / (2 * n)
)

lower_band = np.maximum(F_n - epsilon, 0)
upper_band = np.minimum(F_n + epsilon, 1)

Plot:

plt.step(sorted_residuals, F_n, where="post", label="ECDF")
plt.step(sorted_residuals, lower_band, where="post", label="Lower")
plt.step(sorted_residuals, upper_band, where="post", label="Upper")
plt.xlabel("Residual")
plt.ylabel("CDF")
plt.legend()
plt.show()
```

Change for confidence level

```
# 95%
delta = 0.05

# 99%
delta = 0.01
```

PART F - LOGISTIC REGRESSION / BINARY CLASSIFICATION

13. Basic Logistic Regression

Triggers: target is 0/1, fraud/not fraud, CHD/no CHD, spam/not spam.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train)

# hard 0/1 predictions
pred = model.predict(X_test)
```



```

# probabilities of class 1
prob = model.predict_proba(X_test)[: , 1]

If standardization is required:

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("scaler", StandardScaler()),
    ("logreg", LogisticRegression())
])

model.fit(X_train, y_train)

```

14. Custom threshold

Trigger:

“classify as positive if predicted probability exceeds threshold t.”

```

threshold = 0.40
pred = (prob >= threshold).astype(int)

```

Memory:

```

prob >= threshold -> class 1

```

PART G - CONFUSION MATRIX COUNTS

15. TP, TN, FP, FN

```

TP = np.sum((y_test == 1) & (pred == 1))
TN = np.sum((y_test == 0) & (pred == 0))
FP = np.sum((y_test == 0) & (pred == 1))
FN = np.sum((y_test == 1) & (pred == 0))

```

Memory:

- TP = true 1, predicted 1
 - TN = true 0, predicted 0
 - FP = true 0, predicted 1
 - FN = true 1, predicted 0
-

16. Accuracy / 0-1 loss

```
accuracy = np.mean(pred == y_test)
loss_01 = np.mean(pred != y_test)
```

They satisfy:

accuracy + 0-1 loss = 1

Accuracy higher = better.

0-1 loss lower = better.

17. Precision and Recall for class 1

```
precision_1 = TP / (TP + FP) if (TP + FP) > 0 else 0.0
recall_1 = TP / (TP + FN) if (TP + FN) > 0 else 0.0
```

Memory:

- **Precision:** among predicted positives, how many were truly positive?
 - **Recall:** among actual positives, how many did we find?
-

18. Precision and Recall for class 0

Treat class 0 as the “positive class”:

```
precision_0 = TN / (TN + FP) if (TN + FP) > 0 else 0.0
recall_0 = TN / (TN + FN) if (TN + FN) > 0 else 0.0
```

PART H - COST-SENSITIVE CLASSIFICATION

19. Cost function from TP/TN/FP/FN

Trigger: the question gives numbers like:

- TP cost = 100
- TN cost = 0
- FP cost = 120
- FN cost = 600

Generic template:

```
def cost(y_true, y_prob, threshold,
        cost_TP=100,
```

```

cost_TN=0,
cost_FP=120,
cost_FN=600):

pred = (y_prob >= threshold).astype(int)

TP = np.sum((y_true == 1) & (pred == 1))
TN = np.sum((y_true == 0) & (pred == 0))
FP = np.sum((y_true == 0) & (pred == 1))
FN = np.sum((y_true == 1) & (pred == 0))

total_cost = (
    cost_TP * TP
    + cost_TN * TN
    + cost_FP * FP
    + cost_FN * FN
)

return float(total_cost / len(y_true))

```

Most important exam adaptation

Change ONLY the four costs to match the story.

20. Cost per observation directly

Useful for Hoeffding later:

```

def individual_costs(y_true, y_prob, threshold,
                    cost_TP=100,
                    cost_TN=0,
                    cost_FP=120,
                    cost_FN=600):

    pred = (y_prob >= threshold).astype(int)

    costs = np.zeros(len(y_true), dtype=float)

    costs[(y_true == 1) & (pred == 1)] = cost_TP
    costs[(y_true == 0) & (pred == 0)] = cost_TN
    costs[(y_true == 0) & (pred == 1)] = cost_FP
    costs[(y_true == 1) & (pred == 0)] = cost_FN

    return costs

```

PART I - FIND OPTIMAL THRESHOLD

21. Threshold that minimizes COST

Triggers:

- “check thresholds between 0 and 1”
- “0.01 increments”
- “choose threshold minimizing average cost”

```
thresholds = np.arange(0.00, 1.01, 0.01)
```

```
costs = np.array([
    cost(y_val, prob_val, t)
    for t in thresholds
])
```

```
best_index = np.argmin(costs)
best_threshold = thresholds[best_index]
best_cost = costs[best_index]
```

```
print(best_threshold)
print(best_cost)
```

Plot:

```
plt.plot(thresholds, costs)
plt.xlabel("Threshold")
plt.ylabel("Average cost")
plt.show()
```

22. Threshold that minimizes 0-1 loss

```
losses_01 = []
```

```
for t in thresholds:
    pred_t = (prob_val >= t).astype(int)
    loss_t = np.mean(pred_t != y_val)
    losses_01.append(loss_t)
```

```
losses_01 = np.array(losses_01)
```

```
best_01_index = np.argmin(losses_01)
best_threshold_01 = thresholds[best_01_index]
best_loss_01 = losses_01[best_01_index]
```

Important

The **cost-optimal threshold** can differ from the **accuracy/0-1-loss-optimal threshold** because mistakes can have unequal costs.

23. Difference in business cost between two thresholds

```
cost_at_01_threshold = cost(
    y_val,
    prob_val,
    best_threshold_01
)

cost_difference = (
    cost_at_01_threshold
    - best_cost
)
```

If the question asks for a positive difference, subtract in the order instructed.

PART J - APPLY CHOSEN THRESHOLD TO TEST DATA

24. Validation chooses threshold, TEST evaluates it

```
# threshold selected from validation data
selected_threshold = best_threshold

# probabilities on test set
prob_test = model.predict_proba(X_test)[: , 1]

# final test predictions
pred_test = (prob_test >= selected_threshold).astype(int)

# test cost
test_cost = cost(
    y_test,
    prob_test,
    selected_threshold
)
```

Exam rule

Do not choose the threshold again using the test data if the exam gave you a validation set for threshold selection.

PART K - Hoeffding Confidence Intervals

25. Generic Hoeffding template for bounded values [a,b]

If observations/losses satisfy:

$a \leq Z_i \leq b$

then:

```
mean_Z = np.mean(Z)
n = len(Z)
delta = 0.05

value_range = b - a

epsilon = value_range * np.sqrt(
    np.log(2 / delta) / (2 * n)
)

lower = mean_Z - epsilon
upper = mean_Z + epsilon
```

26. Hoeffding for 0-1 loss

0-1 loss is bounded in [0,1].

```
losses = (pred_test != y_test).astype(float)

mean_loss = np.mean(losses)
n = len(losses)
delta = 0.01 # 99% confidence

epsilon = np.sqrt(
    np.log(2 / delta) / (2 * n)
)

interval = (
```

```

    max(0.0, mean_loss - epsilon),
    min(1.0, mean_loss + epsilon)
)

```

27. Hoeffding for bounded business cost

Suppose costs are:

TP=100, TN=0, FP=120, FN=600

then per-observation cost is bounded in:

[0,600]

```

cost_values = individual_costs(
    y_test,
    prob_test,
    selected_threshold,
    cost_TP=100,
    cost_TN=0,
    cost_FP=120,
    cost_FN=600
)

mean_cost = np.mean(cost_values)
n = len(cost_values)
delta = 0.05

cost_range = 600.0

epsilon = cost_range * np.sqrt(
    np.log(2 / delta) / (2 * n)
)

lower = mean_cost - epsilon
upper = mean_cost + epsilon

```

Critical adaptation

If the largest possible cost changes to 300, 800, etc., change the range accordingly.

28. Hoeffding for CLIPPED regression error

If the question says:

“absolute errors are capped at 80; construct a 99% Hoeffding interval.”

```
errors = np.abs(y_test - y_pred)
clipped_errors = np.minimum(errors, 80)

mean_error = np.mean(clipped_errors)
n = len(clipped_errors)
delta = 0.01

epsilon = 80 * np.sqrt(
    np.log(2 / delta) / (2 * n)
)

lower = mean_error - epsilon
upper = mean_error + epsilon
```

PART L - PROBABILITY CALIBRATION (2024-style backup)

29. Custom logistic/proportional model -> calibration set

The older 2024 papers use a custom logistic model and then calibrate the probabilities using `DecisionTreeRegressor`.

After fitting the classifier:

```
problem2_ps.fit(X_train, y_train)
```

Get its probability-like outputs on calibration data:

```
X_pred_calib = problem2_ps.predict(X_calib).reshape(-1, 1)
```

Fit calibrator:

```
from sklearn.tree import DecisionTreeRegressor
```

```
calibrator = DecisionTreeRegressor()
calibrator.fit(X_pred_calib, y_calib)
```

Use on test:

```
raw_test_prob = problem2_ps.predict(X_test).reshape(-1, 1)
final_prob = calibrator.predict(raw_test_prob)
```

Then convert probabilities to decisions, commonly at 0.5 unless another threshold is specified:


```
final_pred = (final_prob >= 0.5).astype(int)
loss_01 = np.mean(final_pred != y_test)
```

30. Custom logistic negative log-likelihood loss (2024-style backup)

For logistic model:

$P(Y=1|X) = \text{sigmoid}(\text{beta0} + X \text{ beta})$

A stable average negative log-likelihood template:

```
def logistic_loss(X, y, coeffs):
    beta0 = coeffs[0]
    beta = coeffs[1:]

    z = beta0 + X @ beta

    # stable logistic log-loss
    loss = np.mean(
        np.logaddexp(0, z) - y * z
    )

    return float(loss)
```

If the exam provides a class with `loss(...)`, insert this logic there.

PART M - COMPLETE LINEAR REGRESSION TEMPLATE

31. Full linear-regression branch

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# 1. Load
df = pd.read_csv("data.csv")

features = ["x1", "x2", "x3"]
target = "y"
```

```

X = df[features].to_numpy(dtype=float)
y = df[target].to_numpy(dtype=float)

# 2. Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    train_size=0.8,
    random_state=42
)

# 3. Optional standardization - TRAIN ONLY
mu = X_train.mean(axis=0)
sd = X_train.std(axis=0)
sd[sd == 0] = 1

X_train_std = (X_train - mu) / sd
X_test_std = (X_test - mu) / sd

# 4. Fit model
model = LinearRegression()
model.fit(X_train_std, y_train)

# 5. Predict
y_pred = model.predict(X_test_std)

# 6. Residuals
residuals = y_test - y_pred

# 7. Metrics
mse = np.mean(residuals**2)
mae = np.mean(np.abs(residuals))

ss_res = np.sum(residuals**2)
ss_tot = np.sum((y_test - y_test.mean())**2)
r2 = 1 - ss_res / ss_tot

# 8. Baseline
baseline_pred = np.full(len(y_test), y_train.mean())
baseline_mse = np.mean((y_test - baseline_pred)**2)

# 9. Print
print("MSE:", mse)
print("MAE:", mae)
print("R2:", r2)
print("Baseline MSE:", baseline_mse)

```

PART N - COMPLETE LOGISTIC THRESH- OLD/COST TEMPLATE

32. Full logistic-cost branch

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# 1. Load
df = pd.read_csv("data.csv")
X = df[["x1", "x2", "x3"]].to_numpy(dtype=float)
y = df["Class"].to_numpy(dtype=int)

# 2. Train / validation / test
X_train, X_tmp, y_train, y_tmp = train_test_split(
    X, y,
    train_size=0.6,
    random_state=42
)

X_val, X_test, y_val, y_test = train_test_split(
    X_tmp, y_tmp,
    train_size=0.5,
    random_state=42
)

# 3. Fit logistic regression
model = LogisticRegression()
model.fit(X_train, y_train)

# 4. Probabilities
prob_val = model.predict_proba(X_val)[:, 1]
prob_test = model.predict_proba(X_test)[:, 1]

# 5. Cost function - CHANGE COSTS TO MATCH QUESTION
def cost(y_true, y_prob, threshold):
    pred = (y_prob >= threshold).astype(int)

    TP = np.sum((y_true == 1) & (pred == 1))
    TN = np.sum((y_true == 0) & (pred == 0))
```

```

FP = np.sum((y_true == 0) & (pred == 1))
FN = np.sum((y_true == 1) & (pred == 0))

total = 100*TP + 0*TN + 120*FP + 600*FN
return float(total / len(y_true))

# 6. Search threshold on VALIDATION data
thresholds = np.arange(0.00, 1.01, 0.01)

costs = np.array([
    cost(y_val, prob_val, t)
    for t in thresholds
])

best_idx = np.argmin(costs)
best_threshold = thresholds[best_idx]
best_cost_val = costs[best_idx]

# 7. Predictions at optimal threshold
pred_val = (prob_val >= best_threshold).astype(int)

# 8. Precision / recall
TP = np.sum((y_val == 1) & (pred_val == 1))
TN = np.sum((y_val == 0) & (pred_val == 0))
FP = np.sum((y_val == 0) & (pred_val == 1))
FN = np.sum((y_val == 1) & (pred_val == 0))

precision_1 = TP / (TP + FP) if TP + FP > 0 else 0
recall_1 = TP / (TP + FN) if TP + FN > 0 else 0

precision_0 = TN / (TN + FN) if TN + FN > 0 else 0
recall_0 = TN / (TN + FP) if TN + FP > 0 else 0

# 9. Best threshold for 0-1 loss
losses_01 = []

for t in thresholds:
    pred_t = (prob_val >= t).astype(int)
    losses_01.append(np.mean(pred_t != y_val))

losses_01 = np.array(losses_01)
best_threshold_01 = thresholds[np.argmin(losses_01)]

# 10. Cost difference
cost_difference = (
    cost(y_val, prob_val, best_threshold_01)

```

```

        - best_cost_val
    )

# 11. Final test cost using threshold selected on validation
final_test_cost = cost(
    y_test,
    prob_test,
    best_threshold
)

print("Cost-optimal threshold:", best_threshold)
print("Validation cost:", best_cost_val)
print("0-1-optimal threshold:", best_threshold_01)
print("Cost difference:", cost_difference)
print("Final test cost:", final_test_cost)

```

PART O - LAST-MINUTE RULES

1. **Continuous y -> Linear Regression. Binary 0/1 y -> Logistic Regression.**
2. `predict()` gives hard predictions; `predict_proba()[:,1]` gives probability of class 1.
3. If the question says “threshold”, do not automatically use 0.5.
4. If costs are unequal, the cost-optimal threshold may be very different from the accuracy-optimal threshold.
5. Use **training data** to fit the model.
6. Use **validation data** to select a threshold when validation data are provided.
7. Use **test data** for the final evaluation.
8. Standardization mean/std must come from training data only when leakage matters.
9. Regression errors: **MSE/MAE/MARE lower = better.**
10. Classification: **accuracy/precision/recall higher = better; 0-1 loss/cost lower = better.**
11. Hoeffding needs a known bounded range `[a,b]`.
12. 0-1 loss is automatically bounded in `[0,1]`.
13. For business cost, the Hoeffding range comes from the minimum and maximum per-sample costs in the question.
14. 95% confidence -> `delta=0.05`; 99% -> `delta=0.01`.
15. If the question asks for **ECDF confidence bands**, think **DKW**, not Hoeffding.
16. Read the later subquestions before coding: they tell you whether the model branch ends in **metrics, baseline, threshold, cost, calibration,**

DKW, or Hoeffding.